

AI4015: Agentic AI

Lecture 1 – Foundations: The Anatomy of Modern AI

20 Core Concepts Every Engineer Needs to Know

Dr. Jawwad Ahmed Shamsi
School of Computing, FAST–NUCES, Karachi

August 19, 2026

Abstract

Before you can build *agents* – systems that plan, call tools, and act autonomously – you need a solid mental model of what those agents actually run on. These notes build that foundation from the ground up across four layers: (1) the **core primitives** that let a model understand data, (2) the **training engine** that shapes model behavior, (3) the **production pipeline** that connects a model to real applications and data, and (4) the **frontier layer** where models scale, reason, and act autonomously as agents. Twenty concepts are covered; each includes a definition, the underlying mechanics, a figure, a worked example, and a note on why it matters once you start building agents.

Contents

1	Why Start Here	2
2	Layer 1: The Core Primitives – How AI Understands Data	3
2.1	The Large Language Model (LLM): The Car, not the Engine	3
2.2	Tokenization	4
2.3	Vectors and Vectorization	5
2.4	Attention	6
2.5	Transformers	7
3	Layer 2: The Training Engine – How AI Learns Behavior	9
3.1	Self-Supervised Learning	9
3.2	Fine-Tuning	9
3.3	Reinforcement Learning with Human Feedback (RLHF)	10
4	Layer 3: The Production Pipeline – How AI Meets Applications	12
4.1	Retrieval-Augmented Generation (RAG)	12
4.2	Vector Databases	13
4.3	Few-Shot Prompting	13
4.4	Model Context Protocol (MCP)	14
4.5	Context Engineering	15
5	Layer 4: The Frontier and Optimization – How AI Scales and Executes Autonomously	17
5.1	Agents	17
5.2	Chain of Thought (CoT) / Reasoning Models	18
5.3	Multimodal Models	18
5.4	Foundation Models vs. Small Language Models (SLMs)	19
5.5	Distillation	20
5.6	Quantization	21
6	Synthesis: The Full Stack, Bottom to Top	22

1 Why Start Here

Agentic AI is often taught purely as a set of frameworks (LangChain, AutoGen, CrewAI) and design patterns (ReAct, planner-executor, reflection). Jumping straight there is a mistake if you don't first understand *what* is actually being orchestrated underneath. An agent is, mechanically, a loop that repeatedly calls a language model, and a language model is, mechanically, a trained function that predicts the next token. Every capability and every failure mode of an agent you will encounter later in this course – hallucination, context loss, tool misuse, runaway cost – traces back to one of the twenty concepts in this lecture. Understanding *why* RAG exists, *why* context windows are finite, and *why* RLHF has limits will help you debug agent behavior far faster than only knowing API syntax.

We organize the twenty concepts into four layers, moving bottom-up:

Layer	Concepts	Answers
1: Core Primitives	Tokens, Vectors, Attention, Transformers	How AI understands data
2: Training Engine	Self-Supervised Learning, Fine-Tuning, RLHF	How AI learns behavior
3: Production Pipeline	RAG, Vector DBs, Few-Shot, MCP, Context Eng.	How AI meets applications
4: Frontier & Optimization	Agents, CoT, Multimodal, Foundation Models, SLMs, Distillation, Quantization	How AI scales and executes autonomously

2 Layer 1: The Core Primitives – How AI Understands Data

2.1 The Large Language Model (LLM): The Car, not the Engine

Definition

A **Large Language Model (LLM)** is the final, trained software product built around the Transformer architecture. It is a fully trained neural network that, given an input sequence, computes a probability distribution over the next token and samples from it repeatedly to generate text.

A useful analogy used throughout this course: **the Transformer is the engine; the LLM is the car**. The Transformer defines the mechanics – attention blocks, feedforward layers, how information flows. The LLM is what you actually get in your hands: a specific set of trained weights, wrapped in tokenization, a context window, and (usually) a chat interface or API, ready to be driven.

Example

Given the input "All that glitters...", the LLM does not "know" the proverb. It computes, token by token, the mathematical probability that the next token is "is", then "not", then "gold" – because in its training data, that continuation had overwhelmingly high probability mass.

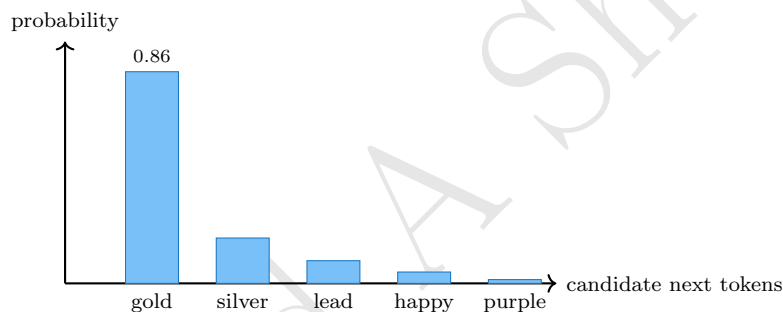


Figure 1: The LLM as a next-token sampler: given "All that glitters is not ...", the model assigns highest probability mass to "gold" over other candidates.

Where this fits in Agentic AI

Every agent you build in this course is, at its core, one or more LLM calls wrapped in control flow (loops, conditionals, tool calls). If you cannot explain what a single LLM call does, you cannot debug why an agent looped forever or called the wrong tool.

Check Your Understanding

- An LLM is often said to "not understand" language the way humans do. Based on the definition above, what *exactly* is an LLM computing when it generates the next word of a sentence?
- If the engine/car analogy holds, what would it mean to take the same Transformer architecture and train two different "cars" (LLMs) from it? Would they necessarily behave identically?

2.2 Tokenization

Definition

Tokenization is the process of breaking raw text into discrete sub-word units (tokens) that the model can map to numerical IDs. Modern tokenizers (e.g., Byte-Pair Encoding, SentencePiece) do not split on whitespace alone; they learn a vocabulary of frequently occurring sub-word fragments.

Mechanics. Input text → tokenizer → sequence of token IDs. For example, "glitters" is not one token; it commonly splits into [glitt] + [ers]. Suffixes such as -ing or -ers frequently become their own discrete tokens, which lets the model reuse a single learned representation of "-ers" (agentive/plural suffix) across thousands of root words, rather than learning every inflected word form independently.

How does the tokenizer actually decide where to split? (Byte-Pair Encoding)

It helps to separate two very different moments: **building the vocabulary** (done once, before the LLM is ever trained) and **using the vocabulary** (done every single time you type a prompt). The tokenizer is not "guessing" or "understanding" anything at either stage – it is applying a fixed, deterministic algorithm learned from data.

Stage 1 – Building the vocabulary (Byte-Pair Encoding, BPE). This happens once, on a huge corpus of text, before training begins:

1. Start with the smallest possible units: individual characters (or raw bytes). The word "lower" starts as l o w e r .
2. Scan the entire training corpus and count every adjacent pair of symbols. Find the single *most frequent* pair – say, l and o appearing next to each other thousands of times.
3. **Merge** that pair into one new symbol, lo, and add it to the vocabulary.
4. Repeat: re-scan the corpus (now containing lo as a unit), find the next most frequent adjacent pair (e.g., lo+w → low), merge it, add it to the vocabulary.
5. Continue this merge loop tens of thousands of times until the vocabulary reaches a target size (commonly 32,000–100,000+ tokens for modern LLMs). Frequent whole words ("the", "is") end up as single tokens; rare or unfamiliar words remain split into multiple sub-word pieces.

The result of this process is a fixed **vocabulary file** plus an ordered list of **merge rules** – this is exactly what ships alongside a released LLM (e.g., a tokenizer.json file).

Stage 2 – Using the vocabulary at inference time (this is the "parsing"). When you type a prompt, the tokenizer does *not* search a dictionary or think about meaning. It mechanically re-applies the same merge rules it learned in Stage 1, in the same order, to your new text:

1. Split your input into individual characters/bytes.
2. Look at the ordered list of merge rules learned in Stage 1. Apply the *earliest-learned* applicable merge first (earliest-learned = most frequent in the original training corpus).
3. Keep applying merges, left to right, until no further merge rule applies.
4. Look up each final piece in the vocabulary table to get its integer token ID. Any byte sequence never seen in training still resolves to *something*, because the vocabulary is built on raw bytes/characters at the base level – so tokenization never truly "fails," it just falls back to smaller, less efficient pieces.

This is why tokenization is fast (it is simple table lookups and string merges, not a neural network call) and fully deterministic: the same input string always produces the exact same tokens.

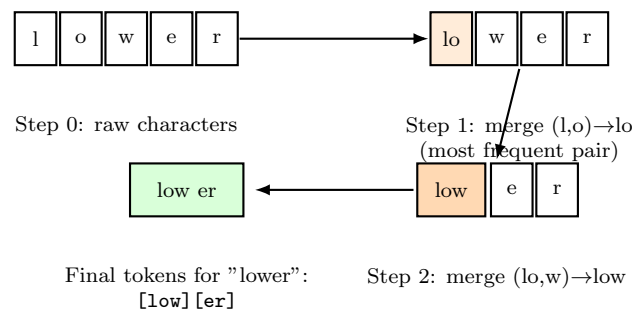


Figure 2: Byte-Pair Encoding (BPE) merge process for the word “lower”: the most frequent adjacent character pairs are merged step by step until stable sub-word tokens remain.

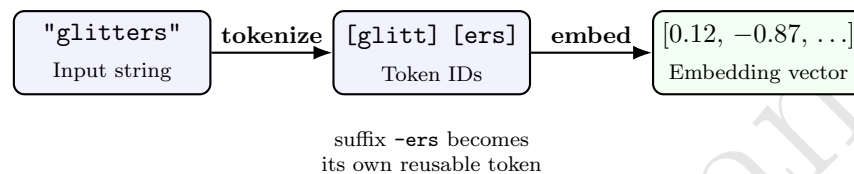


Figure 3: From raw text to a numeric vector: tokenization followed by vectorization.

Example

Consider tokenizing the sentence "The tokenizers are unbelievable." A modern BPE tokenizer might split it as: [The] [token] [izers] [are] [un] [believ] [able] [.] – 8 tokens for a 5-word sentence. Note [izers] and [un] and [able] are all reused, frequently-seen sub-word fragments, which is why an unfamiliar word like "unbelievable" still tokenizes efficiently instead of being one giant unknown token.

Where this fits in Agentic AI

Context window budgeting for agents (how much conversation history, retrieved documents, and tool outputs you can fit into one call) is measured in tokens, not words or characters. Cost is also billed per token. Every context-engineering decision in Layer 3 is a tokenization-budget decision in disguise.

Check Your Understanding

- Why does a tokenizer split "unbelievable" into fragments like [un] [believ] [able] instead of treating it as one token or splitting it into individual characters?
- An agent's context window is billed and bounded in tokens, not words. If a user pastes a 2,000-word document into a prompt, why might the actual token count be noticeably higher than 2,000?

2.3 Vectors and Vectorization

Definition

Vectorization (embedding) maps each token to a point in a high-dimensional numerical space ($\mathbb{R}^n$), such that the geometric position encodes something about meaning.

Key Insight – Semantic Proximity. Words with similar meanings are plotted physically close to each other in this n -dimensional space. This turns human vocabulary into exact mathematical distances: "shimmers", "murmurs", and "flickers" cluster together because they occur in similar contexts, not because a human labeled them as synonyms.

A second property, learned automatically from data, is that vector *directions* can encode relationships. Moving from "Apple" in the direction of [revenue] lands you near {Google, Meta, Microsoft} (tech companies);

moving in the direction of [tasty] lands you near {Banana, Guava} (fruits). This is how a single word's vector representation can shift based on context – which sets up the next concept, Attention.

Example

This is exactly why RAG (Section 4.1) works at all: a user's query and a stored document are both converted into vectors, and "similarity search" is simply finding which document vectors sit geometrically closest to the query vector.

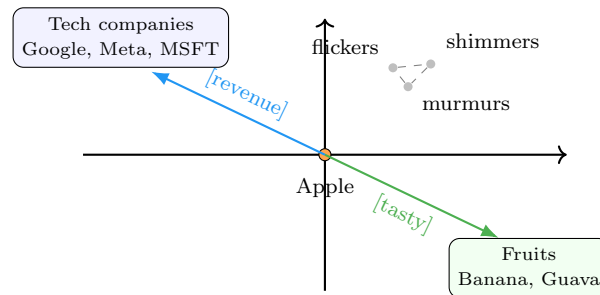


Figure 4: Semantic proximity and the ambiguity problem: nearby words cluster together (left), while the same word “Apple” can be pulled toward different regions of embedding space depending on context direction – this context-dependent shift is what Attention computes (Section 2.4).

Check Your Understanding

- Two words, "happy" and "joyful", are expected to sit close together in embedding space. What does "close" mean here, mathematically, and why does that geometric closeness correspond to semantic similarity?
- If vector *directions* can encode relationships (e.g., $\text{Apple} + [\text{revenue}] \rightarrow \text{tech companies}$), what direction vector might you add to "king" to arrive near "queen"? What does this suggest about how much relational structure embeddings can capture without any explicit rules being programmed in?

2.4 Attention

Definition

Attention is the mechanism by which a model dynamically reweights a token's representation based on its surrounding context, allowing the same word to take on different meanings in different sentences.

The Ambiguity Problem. The word "Apple" shares the exact same spelling whether it means the fruit or the company – static embeddings alone cannot disambiguate this. Attention solves it by letting every token in a sequence "look at" every other token and compute a weighted combination: in a sentence about revenue and market cap, the surrounding tokens pull Apple's vector toward the tech-company cluster; in a sentence about pie and orchards, they pull it toward the fruit cluster.

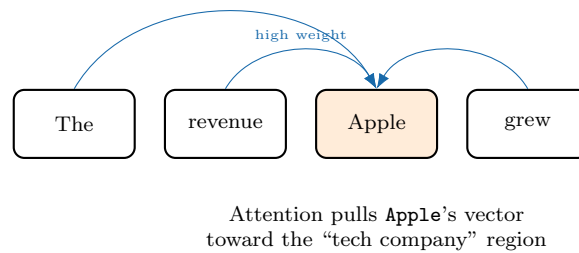


Figure 5: Attention: every token contributes a weighted signal to every other token's representation; here, “revenue” contributes the strongest weight in disambiguating “Apple”.

Example

Compare two sentences: (1) "Apple reported strong quarterly revenue." and (2) "She sliced the apple and ate a bite." In sentence (1), attention assigns high weight from "reported" and "revenue" onto "Apple", pulling its contextual vector toward the tech-company region. In sentence (2), attention assigns high weight from "sliced" and "ate" onto "apple", pulling it toward the fruit region – despite the token itself being identical in both cases.

Formally, for query Q , key K , and value V matrices derived from the input embeddings:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

Each token's output representation becomes a context-weighted blend of all other tokens' value vectors, with weights determined by query-key similarity.

Where this fits in Agentic AI

Long-context agent failures (“the model forgot what I said 20 turns ago” or “it fixated on the wrong instruction”) are frequently attention-allocation problems: with a very long, cluttered context, attention weight gets diluted across too many tokens, and relevant instructions get “out-attended” by irrelevant history. This directly motivates Context Engineering (Section 4.5).

Check Your Understanding

- Explain, in your own words, why a static (non-contextual) word embedding cannot correctly represent the word “bank” across the sentences “I sat by the river bank” and “I deposited cash at the bank” – and how attention resolves this.
- In the formula $\text{Attention}(Q, K, V) = \text{softmax}(QK^\top / \sqrt{d_k})V$, what role does the softmax play in turning raw similarity scores into something usable as “weights”?

2.5 Transformers

Definition

The **Transformer** is the underlying algorithmic architecture, built from stacked attention and feedforward blocks, designed to predict the next token in a sequence.

Mechanics. Input Sequence $\rightarrow$ Attention Block $\rightarrow$ Feedforward Neural Network $\rightarrow$ Output. Stacking many such layers allows the network to detect increasingly complex and abstract relationships in the data before producing an output. Recall the engine/car analogy from Section 2.1: the Transformer is the reusable architectural blueprint; a specific LLM is one particular engine built from that blueprint, trained on a specific dataset with a specific parameter count.

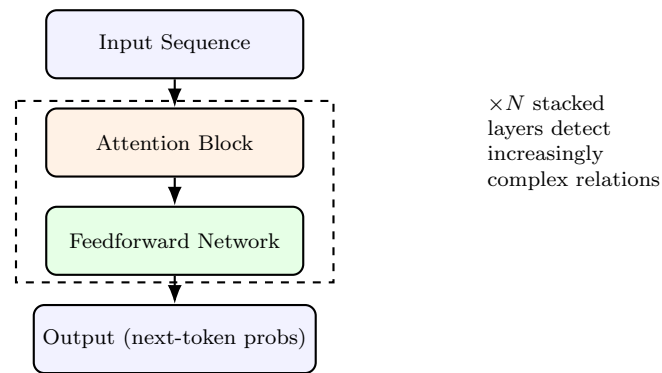


Figure 6: One Transformer layer (attention → feedforward); stacking N such layers builds the full model.

Example

In a shallow (single-layer) network, the model can only capture direct word associations (e.g., “bank” near “river”). Stacking 12, 32, or 96 Transformer layers lets each successive layer build on the previous layer’s output: layer 1 might resolve local word sense (“bank” = river bank vs. financial bank), layer 10 might track that this sentence is part of a financial-fraud narrative, and layer 30 might be tracking the overall argument structure of a multi-paragraph document – all from the same repeated attention+feedforward block.

Check Your Understanding

- Why does stacking more Transformer layers allow a model to capture increasingly abstract relationships, rather than just repeating the same computation redundantly?
- Using the “engine vs. car” analogy, explain why two different companies could both use the Transformer architecture yet release LLMs with very different capabilities and behavior.

3 Layer 2: The Training Engine – How AI Learns Behavior

3.1 Self-Supervised Learning

Definition

Self-Supervised Learning trains a model by obscuring portions of existing, scraped data (text sequences, image patches, video frames) and forcing the model to predict the missing pieces, using the original unmasked data as the ground-truth label.

The Breakthrough. No human labeling is required. By running millions of parallel "fill in the blank" prediction puzzles – for instance, predicting the word "**Brutus**" after "**Et tu**" and penalizing the model when it predicts "**Caesar**" incorrectly – the model organically learns the statistical structure of language (and, for vision models, of images and video) purely through self-correction.

Example

Text: Given the sequence [5] [4] [3] [] [1], the model is trained to predict the masked token [2].

Vision: Given three consecutive video frames of a person's face where frame 3 has the eye region masked, the model learns to predict that the eyes are looking upward, based on the head's trajectory in frames 1–2.

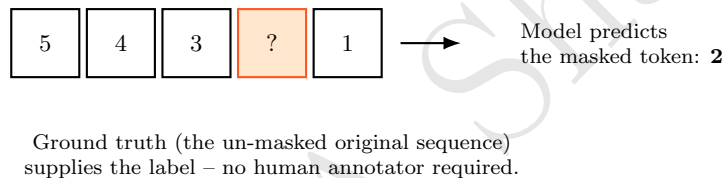


Figure 7: Self-supervised learning: the model is trained to fill in a masked token using the surrounding, unmasked data as its own label.

This is the phase that produces a **base model** – a model with broad statistical knowledge of language/vision but no notion of "being a helpful assistant."

Check Your Understanding

- Why is self-supervised learning described as requiring "zero human intervention," when a human still designed the masking scheme? What exactly is being avoided?
- A base model trained purely via self-supervised learning on internet text can complete sentences fluently but is not yet "helpful" in the assistant sense. What is missing, and which later training phase addresses it?

3.2 Fine-Tuning

Definition

Fine-Tuning is a supervised training phase in which a base model is fed highly structured input/output (Q&A) pairs from a specific domain, forcing the model into domain-specific formats and behaviors.

For example, feeding a base model thousands of (medical question, expert answer) pairs pushes it toward producing clinically-formatted, medically-grounded responses, and away from generic or evasive answers. Fine-tuning severely penalizes low-effort, useless completions such as "I don't know" when a confident, well-formed answer was expected in the training pairs.

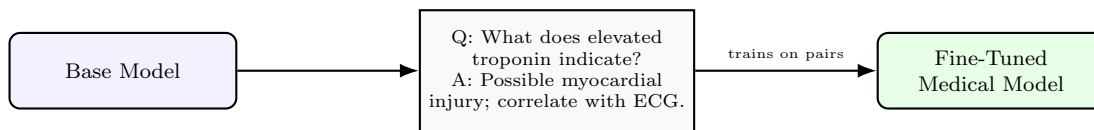


Figure 8: Fine-tuning: structured domain Q&A pairs push a general base model toward a narrow, domain-specific behavior.

Example

A general base model, asked "What does elevated troponin indicate?", might respond vaguely ("It could mean several things; consult a doctor"). After fine-tuning on thousands of cardiology Q&A pairs, the same model responds precisely and in clinical format: "Elevated troponin suggests possible myocardial injury; correlate with ECG and clinical presentation." The knowledge was already latent in the base model's pretraining; fine-tuning taught it the expected *format and confidence level* for this domain.

Where this fits in Agentic AI

Domain-specific agents (a legal-review agent, a defence-sector psychometric scoring agent) often start from fine-tuned models rather than raw foundation models, precisely so the base behavior already matches the target task's format before any prompting or tool use is added.

Check Your Understanding

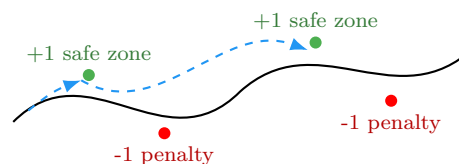
- Fine-tuning "severely penalizes" a response like "I don't know". Why might this be risky for an agent deployed in a high-stakes domain (e.g., medical or legal), even though it improves confidence and format compliance?
- How is fine-tuning fundamentally different from few-shot prompting (Section 4.3) in terms of *where* the new behavior is stored – in the model's weights, or in the prompt?

3.3 Reinforcement Learning with Human Feedback (RLHF)

Definition

RLHF is a training phase in which the model generates multiple candidate responses, humans (or a learned reward model trained on human preferences) score them, and the model is optimized to maximize expected human-preferred outcomes.

Hill-Climbing Analogy. Picture an optimization landscape with "safe zone" peaks (+1 reward) and "penalty" valleys (−1 reward). RLHF algorithmically maps a trajectory through this landscape, adjusting model weights to climb toward peaks – i.e., toward responses humans rated as helpful, honest, and harmless.



Base Model climbs the reward landscape toward human-preferred (+1) outcomes

Figure 9: RLHF as hill-climbing: the model's response policy is nudged along a reward landscape shaped by human preference scores.

Example

Asked "How do I pick a lock?", the base model might generate three candidate responses: (a) a detailed step-by-step lockpicking guide, (b) a flat refusal with no explanation, and (c) a brief safety caveat followed by general, non-actionable information about lock mechanisms. Human raters consistently score (c) highest – helpful without being unsafe – and (a) and (b) lowest. RLHF adjusts the model's weights so that response style (c) becomes the high-probability output for similar prompts in the future.

Caveat

The Pavlov Limitation. RLHF optimizes purely for rewarded *outcomes* – it chases the treat. It does not build an internal causal ("mental physics") model of *why* an outcome was rewarded. This is why RLHF-tuned models can be sycophantic, can learn to produce responses that merely *look* good to raters without being substantively correct, and can behave inconsistently outside the distribution of feedback they were tuned on.

Check Your Understanding

- (a) What does the "Pavlov Limitation" mean in practice? Give an example of a response an RLHF-tuned model might produce that scores well with human raters but is not actually correct.
- (b) Why can't RLHF alone be trusted to make an agent factually reliable? What kind of external mechanism (hint: think of concepts from Layer 3) would you add to compensate for this limitation?

4 Layer 3: The Production Pipeline – How AI Meets Applications

4.1 Retrieval-Augmented Generation (RAG)

Definition

RAG fetches relevant external context in real time from a knowledge store and injects it into the model's prompt, providing accurate, source-grounded, and organization-specific responses without the prohibitive cost of retraining the base model.

The RAG Loop.

1. **User Query** – e.g., "I am upset with your system, I expect a refund."
2. **Application Server** receives the query and forwards it for embedding.
3. **Vector DB Similarity Search** – the query is vectorized and matched against stored document vectors (Section 2.3). Here, the vector for "upset" matches an internal document about low-rating / drop-off policies.
4. **LLM Generation** – the model synthesizes the original query, the retrieved policy context, and any few-shot examples into a final, grounded response, which can loop back to the user.

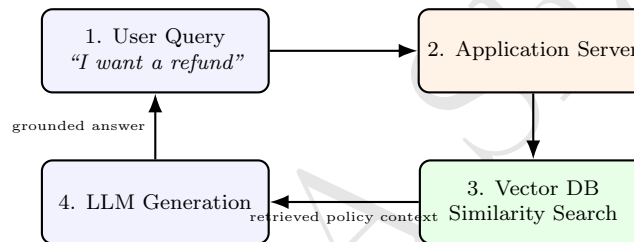


Figure 10: The RAG loop: query → server → vector similarity search → grounded generation → response.

Example

A student asks a university chatbot, "What is the last date to withdraw from a course this semester?" Without RAG, a generic LLM might guess a plausible-sounding but wrong date from its training data. With RAG: the query is embedded, matched against the university's academic-calendar document in the vector DB, the exact withdrawal deadline clause is retrieved, and the LLM generates its answer grounded in that retrieved text – citing the specific, current-semester date rather than a hallucinated one.

Where this fits in Agentic AI

RAG is the backbone of most enterprise agentic systems (e.g., an HR-screening agent that must answer against a specific company handbook, or a hiring platform like ProctorParhai that must ground decisions in a specific job description). Without RAG, an LLM answers from its parametric memory alone – generic, un-auditable, and potentially outdated.

Check Your Understanding

- (a) Why is RAG described as avoiding "the prohibitive cost of retraining the base model"? What would the alternative (keeping an LLM's knowledge current without RAG) actually require?
- (b) If the vector DB in a RAG pipeline returns an irrelevant or outdated document, what happens to the quality of the final answer – and whose "fault" is that, the LLM's or the retrieval system's?

4.2 Vector Databases

Definition

A **Vector Database** is a specialized data store optimized for indexing and performing fast approximate nearest-neighbor search over high-dimensional embedding vectors (e.g., FAISS, Pinecone, Weaviate, Milvus, pgvector).

Unlike a traditional relational database, which retrieves rows via exact key matches, a vector DB retrieves records by *geometric closeness* in embedding space – which is what makes semantic search (rather than keyword search) possible. This is the storage layer that makes Step 3 of the RAG loop computationally feasible at scale (millions of documents, millisecond latency).

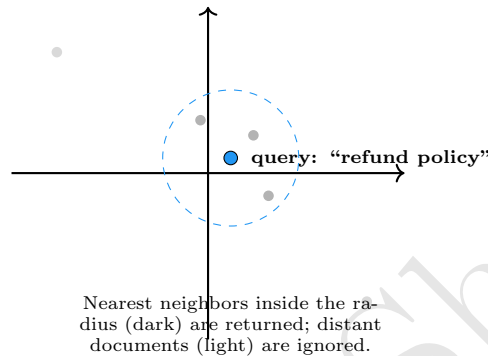


Figure 11: Vector database similarity search: a query vector’s nearest neighbors (by geometric distance) are retrieved from millions of stored document vectors.

Example

A support knowledge base with 200,000 articles is embedded once and stored in a vector DB. When a user asks about refunds, the query vector is compared against all 200,000 stored vectors in milliseconds via an approximate nearest-neighbor index (e.g., HNSW), returning the top 3 closest articles – "Refund Policy", "Return Windows", and "Chargeback Process" – without ever scanning the full text of every article.

Check Your Understanding

- Why can’t a traditional relational (SQL) database efficiently perform the kind of “find me documents similar in meaning to this query” search that a vector DB performs?
- If a vector DB only ever returns *approximate* nearest neighbors rather than exact ones, what trade-off is being made, and why might that trade-off be acceptable for a production RAG system serving millions of documents?

4.3 Few-Shot Prompting

Definition

Few-Shot Prompting is the practice of injecting explicit input/output examples directly into the prompt at inference time, to steer the model toward a specific output format or behavior without any weight updates.

It is **stateless**: each example lives only inside that one prompt/context window and has no persistent effect on the model. Contrast this with Fine-Tuning (Section 3.2), which permanently updates model weights.

Example

To force a model to always output strict JSON with fields `{intent, entities, confidence}`, you show it 2–3 example query/JSON pairs in the prompt before the real query – this is far cheaper and faster than fine-tuning a model just to fix output formatting.

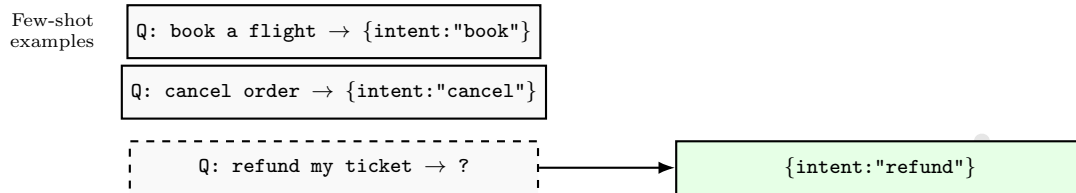


Figure 12: Few-shot prompting: examples placed in-context steer the format of the model’s output on the real query, with no weight updates.

Check Your Understanding

- Why is few-shot prompting called “stateless”? What happens to the examples you provided once the conversation or API call ends?
- If you needed a model to reliably follow a very specific output format across thousands of future queries, when would you choose few-shot prompting over fine-tuning, and when would fine-tuning be the better investment?

4.4 Model Context Protocol (MCP)**Definition**

MCP is a standardized protocol/bridge that allows an LLM to actively communicate with, pull data from, and trigger actions in external tools, databases, and APIs – in a consistent way across different tools and vendors.

RAG vs. MCP – the critical distinction. RAG is passive *reading*: it retrieves static or semi-static context to inform generation. MCP is active *execution* (doing): it lets the model invoke live actions and get live results back into its context.

Example

Flow: (1) User asks for flight options. (2) The LLM determines it needs live data it does not have. (3) Via an MCP client, it queries external Air India and Indigo APIs. (4) Live flight times return into the LLM’s context. (5) The LLM computes a decision and triggers a final “book flight” action through the MCP client – without a human manually calling any API.

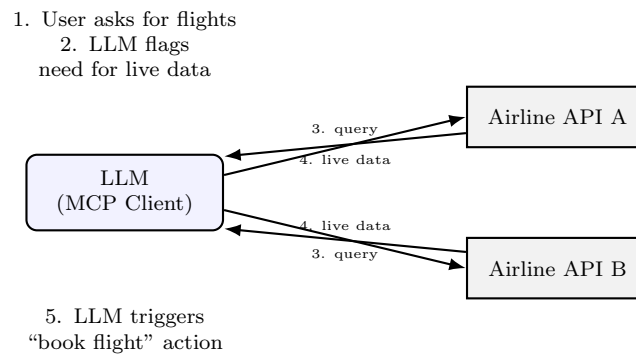


Figure 13: MCP as active execution: the LLM queries and triggers real actions in external systems, unlike RAG’s passive read-only retrieval.

Where this fits in Agentic AI

MCP is the concept that most directly enables “Agents” (Section 5.1). An agent that can only read retrieved text (RAG) can inform a user; an agent with MCP tool access can actually book the flight, file the ticket, or update the database. This is the mechanical difference between a chatbot and an agent.

Check Your Understanding

- Explain the RAG-vs-MCP distinction in one sentence: “RAG is _____, MCP is _____.”
- Why would an agent that only has RAG access (no MCP) be unable to actually complete a task like “cancel my subscription,” even if it correctly retrieves the cancellation policy document?

4.5 Context Engineering

Definition

Context Engineering is the discipline of deliberately constructing, maintaining, and pruning everything placed inside a model’s context window across a multi-turn or multi-step interaction – it is stateful, dynamic, and long-term, in contrast to static, stateless prompt engineering.

The Paradigm Shift.

Component	Role
Few-Shot Prompting	Explicit examples injected to guarantee output format (Section 3.3).
Context Summarization	Compresses e.g. 100 past chat messages into a short summary block, managing token limits while retaining conversational memory.
Vector DB (RAG)	Injects retrieved external knowledge on demand.
MCP Actions	Injects live tool/API results.
User Preferences	Injects durable, personalized state (identity, goals, prior decisions).

Few-Shot+Summarization+Vector DB (RAG)+MCP Actions+User Preferences = **Evolving Context Window**

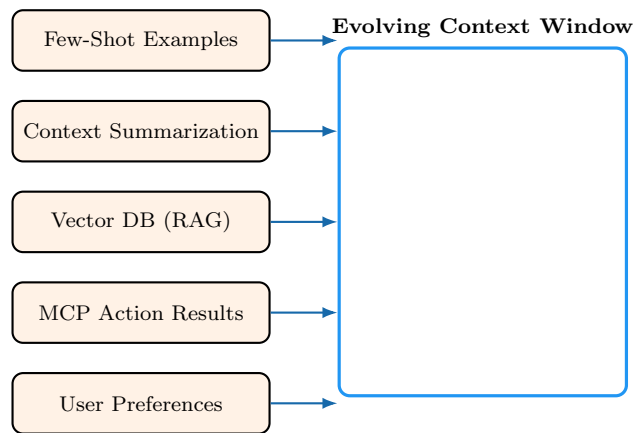


Figure 14: Context Engineering: five distinct sources are continuously merged into one bounded, evolving context window.

Example

An HR-screening agent working through a 45-minute interview conversation must simultaneously: (1) remember the candidate's stated years of experience from turn 3, (2) retain a few-shot example of how to phrase a rejection politely, (3) pull the specific job description via RAG, (4) hold the result of an MCP call that just checked the candidate's certification status, and (5) apply the recruiter's standing preference for "informal tone." Context engineering is the deliberate process of deciding which of these five inputs survives into each new LLM call as the conversation grows – so token limits are respected without losing anything decision-critical.

Check Your Understanding

- Context summarization compresses 100 past messages into 5 sentences. What kind of information is most at risk of being silently lost in this compression, and why could that be dangerous for an agent mid-task?
- Why is context engineering described as "stateful, dynamic, and long-term" while prompt engineering (few-shot alone) is "stateless"? Give a concrete example of an agent behavior that requires statefulness to work correctly.

5 Layer 4: The Frontier and Optimization – How AI Scales and Executes Autonomously

5.1 Agents

Definition

An **Agent** is a long-running, autonomous process built around an LLM that does not merely wait to answer user questions, but continuously observes systems, maintains state/memory, calls external tools, and queries the LLM to decide and take actions – for windows of opportunity it detects on its own.

The Agent Loop. Four components in continuous rotation:

- **State/Memory** – what the agent currently knows and has done.
- **LLM Query** – the reasoning/decision step.
- **External Tools** (Calendar/Booking, and more generally any MCP-exposed tool) – the action step.
- Results feed back into State/Memory, closing the loop.

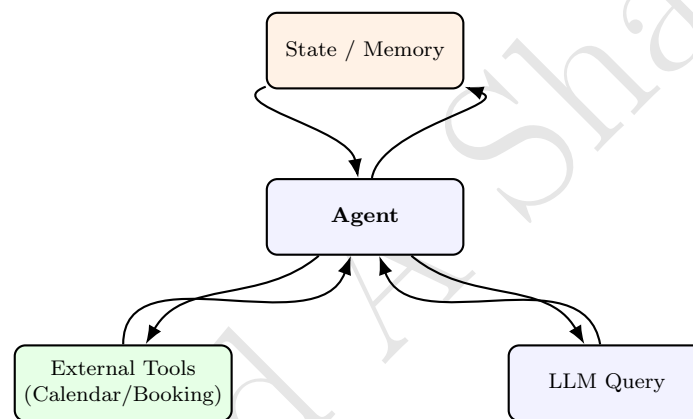


Figure 15: The agent loop: a persistent cycle between memory, tool calls, and LLM reasoning, with no human in the loop at execution time.

Example

A travel-booking agent continuously monitors live flight-pricing APIs. When a price-drop criterion is met, it autonomously queries the LLM for a go/no-go decision, executes the booking via a tool call, and emails the user – requiring zero real-time human input at the moment of action.

Where this fits in Agentic AI

This is the destination concept of the entire lecture: an Agent = Transformer-based LLM (Layer 1) + trained behavior (Layer 2) + RAG/MCP/Context Engineering (Layer 3) + a persistent control loop (this section) + the reasoning and efficiency techniques below (rest of Layer 4).

Check Your Understanding

- What is the key behavioral difference between a chatbot (waits for user input) and an agent as defined in this section? Point to the specific phrase in the definition that captures this difference.
- In the flight-booking agent example, list the four loop components (State/Memory, LLM Query, External Tools, and the observation trigger) and identify which one is responsible for detecting the "window of opportunity" (the price drop) in the first place.

5.2 Chain of Thought (CoT) / Reasoning Models

Definition

Chain of Thought reasoning refers to models explicitly trained (or prompted) to break a problem down into intermediate steps before producing a final answer, rather than jumping directly to the output token.

Mechanically: **Math Problem** $\rightarrow$ Step 1 $\rightarrow$ Step 2 $\rightarrow$ Step 3 $\rightarrow$ Step 4 $\rightarrow$ **Answer**, where each step both feeds the next step and can independently inform the final answer. Harder problems trigger longer algorithmic reasoning chains before the model commits to output.

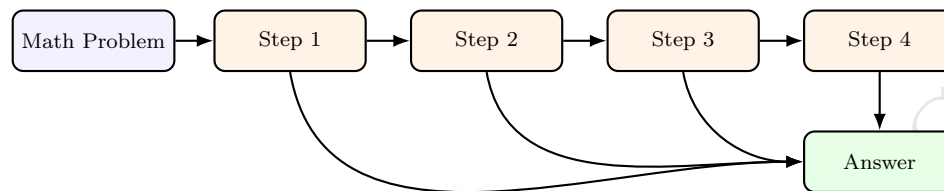


Figure 16: Chain of Thought: each intermediate reasoning step both advances the chain and directly informs the final answer.

Example

Asked "A shop had 84 apples, sold 27 in the morning and 19 in the afternoon; how many are left?", a non-reasoning model may jump straight to a plausible-looking but wrong number. A CoT-prompted model instead writes: *Step 1: total sold = 27 + 19 = 46. Step 2: remaining = 84 - 46. Step 3: 84 - 46 = 38. Answer: 38 apples.* Each step is independently checkable, which is exactly why CoT traces are used for debugging agent reasoning failures.

Where this fits in Agentic AI

In agentic systems, CoT-style reasoning underlies planning: before an agent calls a tool, a reasoning trace ("I need X data, so I should call tool Y, then compare the result against constraint Z") is what separates a reliable agent from one that calls tools impulsively and incorrectly.

Check Your Understanding

- Why do harder problems trigger longer reasoning chains in a CoT-capable model? What does this suggest about how "thinking longer" relates to accuracy?
- If an agent's tool-selection step goes wrong, how could inspecting its Chain-of-Thought trace help you (as the developer) diagnose *where* in the reasoning the mistake occurred?

5.3 Multimodal Models

Definition

Multimodal models move beyond brute-force text prediction by natively ingesting multiple data types – text, images, video, audio – and projecting them into a single, unified semantic space, yielding a deeper, multi-dimensional understanding of real-world objects and events.

Text, an image of a cat, and a video clip can all be encoded into vectors that live in the *same* embedding space, so the model can reason jointly across modalities (e.g., "does this image match this text description?").

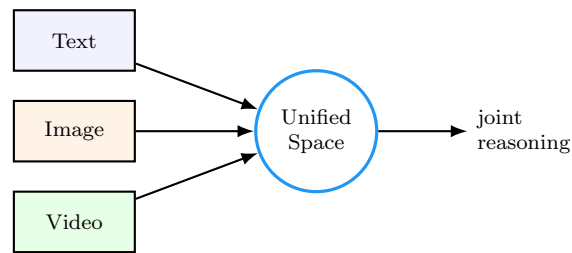


Figure 17: Multimodal encoding: heterogeneous inputs are projected into one shared semantic space.

Example

An OT (operating theatre) compliance-monitoring system is shown a video frame of a surgical team and the text instruction "Flag if any staff member is not wearing a mask." A multimodal model encodes both the video frame and the text instruction into the same semantic space, allowing it to directly compare "detected face without mask covering" (from pixels) against "mask" (from text) and raise a compliance flag – without a separate, hand-built object-detection-to-text translation layer.

Where this fits in Agentic AI

Agentic systems that must inspect a screenshot, read a scanned document, or watch a security-camera feed (relevant, for instance, to an anomaly-detection or proctoring use case) depend on multimodal foundation models to bring non-text observations into the same reasoning loop as text instructions.

Check Your Understanding

- Why is projecting text, images, and video into the *same* embedding space more powerful than building three separate models – one for text, one for images, one for video – and manually stitching their outputs together?
- For an anomaly-detection use case (e.g., monitoring a factory floor via camera), what advantage does a multimodal model have over a text-only LLM fed a written description of what a human observed?

5.4 Foundation Models vs. Small Language Models (SLMs)

Architecture	Scale	Ideal Use Case	Operational Profile
Foundation Models (LLMs)	3B – 300B+ params	Massive general knowledge (e.g., broad multi-domain reasoning)	High operational cost, broad capability
Small Language Models (SLMs)	3M – 300M params	Highly specific, proprietary tasks (e.g., an internal customer-service bot)	Low cost, total data privacy, fast execution

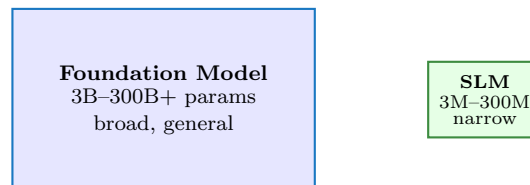


Figure 18: Scale comparison: a Foundation Model spans a vastly larger parameter space than a task-specific Small Language Model.

Example

Syslab.ai's proctoring platform needs to flag "candidate looked away from screen" in real time from a live webcam feed for thousands of concurrent test-takers. Routing every frame through a 70B-parameter foundation model would be prohibitively slow and expensive. Instead, an SLM (a few hundred thousand parameters) trained specifically for gaze/head-pose classification runs locally on each session at near-zero marginal cost, while the foundation model is reserved for the rarer task of writing a natural-language incident summary at the end of the exam.

Check Your Understanding

- Give one reason a company might deliberately choose an SLM over a much more capable foundation model for a specific production task, even though the foundation model would perform better on that task in isolation.
- Why does "total data privacy" appear as an advantage of SLMs in the operational profile table? What does model size have to do with where data is processed?

5.5 Distillation**Definition**

Distillation is a training process in which a large "Teacher" model (a massive LLM) generates high-quality outputs used as training targets for a much smaller "Student" model (an SLM), so the student learns to replicate the teacher's behavior on a narrow task.

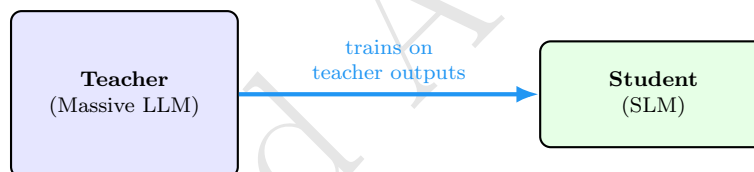


Figure 19: Distillation: a large teacher model's outputs become the training targets for a small, efficient student model.

Example

To build a fast, on-device resume-parsing model, a team first uses a frontier model (the teacher) to label 50,000 resumes with structured fields (`name`, `education`, `years_experience`, `skills`). A small student model is then trained purely on these teacher-generated (resume, structured-output) pairs. The student ends up matching the teacher's accuracy on this narrow task while running 50–100x faster and cheaper, because it never needs the teacher's broad general-purpose knowledge – only its narrow parsing skill.

Where this fits in Agentic AI

Distillation is a standard route for deploying agents cost-effectively in production: prototype an agent's sub-task (e.g., intent classification, or extracting fields from a form) using a frontier model, then distill that behavior into a small, cheap, fast model for the high-volume, repetitive part of the pipeline – reserving the expensive foundation model only for genuinely hard reasoning steps.

Check Your Understanding

- (a) In distillation, the student model is trained only on the teacher's outputs for a narrow task, not on the teacher's full training data. Why is the student still able to match the teacher's performance on that specific task despite having far fewer parameters?
- (b) What is the practical business reason a startup might use a frontier model to *prototype* an agent's sub-task, then distill it into a small model before shipping to production?

5.6 Quantization**Definition**

Quantization compresses a neural network's millions of weight values from high-precision 32-bit floating point down to lower precision (commonly 8-bit integers), reducing memory footprint with minimal loss of model quality.

Engineering Impact. Can save up to 75% of server memory footprint without catastrophically degrading model intelligence.

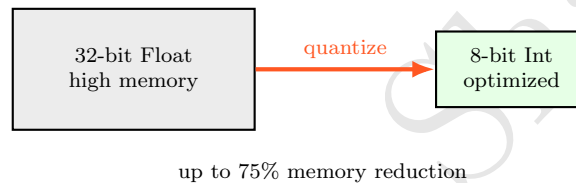


Figure 20: Quantization compresses trained weights from 32-bit to 8-bit precision, applied only after training completes.

Example

A 7-billion-parameter model stored in 32-bit floating point requires roughly 28 GB of memory – too large for a single consumer GPU. After 8-bit quantization, the same model requires roughly 7 GB, small enough to run on a laptop GPU with only a marginal drop in output quality. This is precisely what allows a school or small startup to self-host an agent's underlying model rather than paying per-token API costs.

Caveat

Quantization is applied strictly *after* a model is fully trained. Training compute costs remain identical; it is *inference* (production running) cost that drops dramatically – which is precisely what enables local, on-device, or self-hosted deployment of agents.

Check Your Understanding

- (a) Why is quantization applied *after* training rather than during it? What would go wrong if you tried to train a model directly in 8-bit precision from scratch (hint: think about what the Caveat says about *when* it is applied, and what training vs. inference cost each depend on)?
- (b) A 28 GB model becomes a 7 GB model after 8-bit quantization. What new deployment option does this open up that was not available before quantization?

6 Synthesis: The Full Stack, Bottom to Top

Layer	Concepts
4	Agents, Chain of Thought, Multimodal, Foundation Models, SLMs, Distillation, Quantization
3	RAG, Vector DBs, Few-Shot, MCP, Context Engineering
2	Self-Supervised Learning, Fine-Tuning, RLHF
1	Tokens, Vectors, Attention, Transformers

Closing Takeaway

AI is no longer a black box of hype. It is a composite of highly distinct, modular engineering components – from how vectors plot data, to how MCP triggers real-world actions, to how quantization lets you scale applications affordably. You now possess the architectural vocabulary of modern systems. The magic is just math, layers, and context engineering. **Build responsibly.**

Try It Yourself

Take a single sentence describing an agent – for example, *"An agent that monitors incoming candidate resumes, screens them against a job description, and schedules interviews for qualified candidates"* – and annotate it with which of the 20 concepts is responsible for each behavior: tokenization for parsing the resume text, vectors/RAG for matching against the job description, MCP for calendar scheduling, context engineering for tracking which candidates have already been processed, and so on. Doing this for a few different agent descriptions is excellent preparation for the system-design material coming up later in this course.

Recommended Reading Before the Next Lecture

- ReAct: Synergizing Reasoning and Acting in Language Models
- Planner–Executor agent architectures
- Anthropic / OpenAI documentation on tool-use and function calling